Solving 2048: A Comparative Study of Classical Reinforcement Learning and LLM-Guided RLVR

Shriman Raghav Gautham Ramkumar
MS Robotics, Khoury College of Computer Sciences
Northeastern University, Boston, MA
srinivasan.shrim@northeastern.edu

Abstract

We present a systematic comparison of classical reinforcement learning and Reinforcement Learning with Verifiable Rewards (RLVR) on the 2048 puzzle game. Six classical agents – Deep Q-Network (DQN), Proximal Policy Optimization (PPO), Advantage Actor-Critic (A2C), Quantile Regression DQN (QR-DQN), discrete Soft Actor-Critic (SAC), and Semi-gradient SARSA with linear function approximation – are benchmarked against Qwen2.5 language models (0.5B and 1.5B parameters) fine-tuned via Group Relative Policy Optimization (GRPO). All deep classical agents share an identical three-layer CNN backbone (330K parameters) to isolate algorithmic effects. We evaluate at three levels: 1M training steps, 5M training steps, and unlimited-attempt hunt mode. DQN achieves the highest classical score (avg 7,744 at 5M steps; tile 2048 in hunt mode), while Semi-gradient SARSA with only 120 parameters outperforms all deep on-policy methods, validating Sutton & Barto's thesis that engineered features trump raw network capacity when data is scarce. GRPO-trained LLMs demonstrate a clear model-scale dependence ($3\times$ parameters $\rightarrow 3.3\times$ score), but classical agents remain $\sim 400\times$ faster at inference. We release a live dashboard and complete codebase for reproducibility.

1. Introduction

The tile-sliding puzzle 2048 was introduced in 2014 by Gabriele Cirulli and quickly attracted attention from the AI community. On every turn the player selects one of four cardinal directions; all tiles slide accordingly, identical adjacent tiles merge (doubling their value), and a fresh 2 or 4 tile spawns at a uniformly random empty cell. The game terminates when no legal move remains.

Despite its compact 4×4 board, 2048 poses fundamental challenges for learning agents. The state space is vast ($\sim 10^{19}$ configurations), rewards are sparse and delayed (hundreds of preparatory moves may earn nothing before a single high-value merge), and stochastic tile spawns make planning difficult. These properties make 2048 an ideal benchmark for comparing fundamentally different learning paradigms.

Recently, a new paradigm has emerged: Reinforcement Learning with Verifiable Rewards (RLVR). Instead of training a neural network policy via environment interaction, RLVR fine-tunes large language models (LLMs) to generate chain-of-thought reasoning validated by deterministic reward functions [1, 2]. While RLVR has yielded breakthroughs in mathematical reasoning, its application to spatial strategy games remains largely unexplored.

This paper asks three questions: **(Q1)** How do six classical RL algorithms compare on 2048 when architectural confounds are removed? **(Q2)** Can small LLMs (0.5B–1.5B parameters) acquire 2048 strategy through GRPO alone, without using proprietary LLM APIs? **(Q3)** What are the fundamental trade-offs between classical RL's throughput and RLVR's interpretability?

Our contributions include:

1. A controlled benchmark of six classical agents and two GRPO-LLM agents on a unified 2048 environment with clear 1M/5M/hunt-mode evaluation tiers.
2. Demonstration that Semi-gradient SARSA (Sutton & Barto, Algorithm 10.1 [18]) with 120 parameters outperforms all deep on-policy methods with 330K+ parameters.
3. A GRPO training pipeline for locally deployed LLMs with analysis of model-scale dependence and spatial reasoning failures.
4. Evidence that on-policy methods (PPO, A2C) actively degrade with extended training on 2048, losing 9–12% score from 1M to 5M steps.
5. A live web dashboard for step-by-step replay visualization.

2. Background

2.1 Markov Decision Process Formulation

The 2048 game is modeled as a finite-horizon MDP $(\mathscr{S}, \mathscr{A}, P, R, \gamma)$ where:

- $\mathscr{S}$: Board states, encoded as 16-channel binary tensors $\mathbf{x} \in \{0,1\}^{16\times4\times4}$ for CNN agents, or ASCII text for LLM agents.
- $\mathscr{A} = \{\text{UP}, \text{RIGHT}, \text{DOWN}, \text{LEFT}\}$: Four discrete actions.
- $P(s'|s,a)$: Deterministic tile-sliding with stochastic spawn (90% tile-2, 10% tile-4 at random empty cell).
- $R(s,a,s')$: Score-delta plus milestone bonuses (+50 at 256, +100 at 512, +200 at 1024) and -1 penalty for invalid moves.

- $\gamma = 0.99$: Discount factor.

2.2 Value-Based Methods

Value-based approaches estimate the action-value function $Q^\pi(s,a) = \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t r_t | s_0 = s, a_0 = a]$ and act greedily. DQN [5] parameterizes Q as a neural network and stabilizes learning via experience replay and a frozen target network. QR-DQN [8] extends DQN by modeling the full return distribution via quantile regression.

2.3 Policy-Gradient Methods

Policy-gradient methods directly optimize the policy $\pi_\theta(a|s)$. PPO [6] uses a clipped surrogate objective with Generalized Advantage Estimation (GAE). A2C [7] is a synchronous actor-critic using n-step returns. SAC [9] augments the RL objective with an entropy bonus, encouraging exploration.

2.4 Linear Function Approximation

Sutton & Barto (§9.3, [18]) describe linear function approximation where $\hat{q}(s,a,\mathbf{w}) = \mathbf{w}_a^\top \phi(s)$, with $\phi(s)$ a hand-crafted feature vector. Semi-gradient SARSA (Algorithm 10.1) updates weights using only the gradient of the prediction, not the bootstrap target. Under linear FA, semi-gradient SARSA converges to a TD fixed point with bounded approximation error (§9.4).

2.5 RLVR and Group Relative Policy Optimization

RLVR trains LLMs using rewards from deterministic verifiers rather than human feedback [1]. Group Relative Policy Optimization (GRPO) [12] eliminates the critic network by normalizing advantages within a group of G sampled completions per prompt:

$$\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G + \varepsilon} \tag{1}$$

This replaces the critic's state-dependent baseline with a sample-based, prompt-specific baseline.

3. Related Work

Classical approaches to 2048. Szubert & Jaśkowski [13] use temporal-difference learning with n-tuple networks, achieving strong performance via tabular value approximation over board patterns. Saligram et al. [3] apply DQN with delayed-reward shaping. Our work differs by comparing six algorithms on a shared backbone, isolating the learning algorithm from architectural choices.

Deep RL for games. Mnih et al. [5] demonstrated human-level Atari play with DQN. Subsequent work introduced distributional RL [8], entropy-regularized policies [9], and trust-region methods [6]. We evaluate all four paradigms on a single game with matched architectures.

LLMs for game-playing. Hu et al. [4] benchmark LLMs on diverse games including 2048, finding that even large models struggle with spatial reasoning. Our work extends this by *training* LLMs via GRPO rather than prompting pretrained models, and directly comparing the trained LLM against classical RL agents under controlled conditions.

RLVR. DeepSeek-R1 [1] showed that GRPO can incentivize reasoning in LLMs using verifiable rewards. DeepSeek-Math [12] applied GRPO to mathematical reasoning. We extend RLVR to spatial strategy games and quantify the gap between RLVR and classical approaches.

4. Project Description

4.1 Environment

We implement a custom 2048 Gymnasium [17] environment. For CNN agents, the board is encoded as a 16-channel binary tensor (Eq. 2):

$$x_{k,i,j} = \mathbb{1}[B_{i,j} = 2^k], \quad k = 0, \ldots, 15 \tag{2}$$

For LLM agents, the board is serialized as a labeled ASCII grid with current score and max tile.

4.2 Shared CNN Backbone

Every deep classical agent uses the same three-layer convolutional feature extractor: Conv2D($16 \to 128$, 2×2) $\to$ ReLU $\to$ Conv2D($128 \to 128$, 2×2) $\to$ ReLU $\to$ Conv2D($128 \to 128$, 2×2) $\to$ ReLU $\to$ FC($128 \to 256$). This produces a 256-dimensional feature vector; each algorithm appends its own task-specific heads. The shared backbone ensures performance differences reflect the *learning algorithm*, not network capacity.

4.3 Classical Agents

4.3.1 DQN (Custom PyTorch).

Our DQN uses a custom PyTorch implementation rather than Stable-Baselines3 (SB3) because SB3's DQN lacks direct Q-value masking for invalid actions (it only supports action-space wrappers). Our implementation masks invalid Q-values to $-\infty$ *inside the forward pass*, enabling proper ε-greedy selection over legal moves only.

Hyperparameters: replay buffer 100K, batch size 64, ε: $1.0 \to 0.01$ over 100K steps, target sync every 1K steps, learning rate 10^{-4} (Adam), gradient clipping 1.0.

4.3.2 SAC – Discrete (Custom PyTorch).

Standard SAC [9] targets continuous actions. We implement the discrete adaptation of Christodoulou [10] with categorical policies, twin Q-networks, and automatic temperature tuning. This required custom code because SB3 does not natively support discrete SAC with invalid action masking in the policy distribution.

Hyperparameters: replay buffer 200K, batch size 256, Polyak $\tau = 0.005$, learning rate 10^{-4}, warmup 10K steps, target entropy $0.4 \cdot \log|\mathscr{A}|$.

4.3.3 PPO – MaskablePPO (Stable-Baselines3).

We use SB3's MaskablePPO [14], which natively supports action masking via environment-provided masks.

Hyperparameters: rollout 2048 steps $\times$ 8 parallel envs, batch size 64, 10 epochs per update, learning rate 3×10^{-4}, GAE $\lambda = 0.95$, clip $\varepsilon = 0.2$, entropy coefficient $c_e = 0.01$.

4.3.4 A2C (Stable-Baselines3).

A2C [7] is on-policy actor-critic without PPO's clipping mechanism.

Hyperparameters: 5-step returns, learning rate 7×10^{-4}, entropy coefficient 0.01, value coefficient 0.5.

4.3.5 QR-DQN (SB3-Contrib).

QR-DQN [8] models the return distribution via N=50 quantiles, outputting 4×50=200 values.

Hyperparameters: buffer 100K, batch 64, learning rate 10^{-4}, target sync every 1K steps, ε: $1.0 \to 0.05$ over 10% of training.

4.3.6 Semi-gradient SARSA (Custom NumPy).

Following Sutton & Barto, Algorithm 10.1 [18], we implement episodic semi-gradient SARSA with linear function approximation. The agent maintains $\mathbf{w} \in \mathbb{R}^{4\times30}$ (120 parameters total) mapping a 30-dimensional feature vector $\phi(s)$ to Q-values:

$$\hat{q}(s,a,\mathbf{w}) = \mathbf{w}_a^\top \phi(s) \tag{3}$$

The feature vector $\phi(s)$ encodes 30 dimensions following Sutton & Barto's guidance on feature construction (§9.5): 16 log-tile values, empty ratio, merge potential, 4 monotonicity scores, max-tile indicator, corner bonus, 3 tile distribution statistics, smoothness, snake-pattern score, and edge-tile sum. The semi-gradient update rule (§10.1) differentiates only through the prediction:

$$\mathbf{w}_{a_t} \leftarrow \mathbf{w}_{a_t} + \alpha\delta_t\phi(s_t), \quad \delta_t = r_t + \gamma\hat{q}(s_{t+1},a_{t+1}) - \hat{q}(s_t,a_t) \tag{4}$$

Hyperparameters: step-size $\alpha = 10^{-3}$, ε-decay 0.99995/step, $\varepsilon_{\min} = 0.05$.

4.4 GRPO-Trained LLM Agent

4.4.1 Why Local GRPO, Not an LLM API?

We train LLMs locally via GRPO rather than using proprietary APIs (e.g., OpenAI, Anthropic) for three reasons:

1. **Policy gradient access.** GRPO requires computing per-token log-probabilities $\log\pi_\theta(o_{i,t}|q,o_{i,<t})$ and their gradients to form the importance ratio $r_t^i(\theta) = \pi_\theta/\pi_{\theta_{\text{old}}}$. Commercial APIs do not expose model weights or gradient computation; only logit/log-prob outputs are sometimes available, which is insufficient for the clipped PPO-style policy update.
2. **Cost.** GRPO generates G=4 completions per prompt across 300 training steps, totaling ∼1,200 forward passes per epoch. At API pricing ($0.10–$1.00 per 1K tokens), this training loop would cost $100–$500 per experiment, making iterative hyperparameter tuning infeasible on a student budget.
3. **Reproducibility.** Local deployment with fixed random seeds and quantization settings ensures deterministic training. API-served models may change versions silently, undermining reproducibility.

Table 1: Classical Agent Performance (Last 100 Episodes, 1M Steps)

Agent	Avg Score	Max Score	Max Tile	256+	512+	1024+	Impl.
DQN	**6,384**	**14,050**	**1024**	92%	67%	10%	Custom
SARSA	2,455	10,208	1024	45%	9%	1%	NumPy
A2C	1,205	2,856	256	14%	0%	0%	SB3
SAC	1,132	4,814	512	6%	1%	0%	Custom
PPO	1,038	3,040	256	7%	0%	0%	SB3
QR-DQN	782	2,176	256	2%	0%	0%	SB3

4.4.2 Model and Training.

We fine-tune Qwen2.5-0.5B-Instruct (494M params) and Qwen2.5-1.5B-Instruct (1.8B params) [15] using 4-bit NF4 quantization via Unsloth [16] with LoRA adapters (r=32, α=32). The GRPO loss follows Equation 1 with a KL penalty:

$$\mathscr{L}_{\text{GRPO}}(\theta) = \mathbb{E}_q\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left(L_t^{\text{clip}} - \beta D_{\text{KL}}\right)\right] \tag{5}$$

where $L_t^{\text{clip}} = \min(r_t^i\hat{A}_i, \text{clip}(r_t^i, 1\pm\varepsilon)\hat{A}_i)$ and β=0.04.

4.4.3 Reward Scheduling.

The reward function uses three stages to avoid cold-start collapse: (1) format compliance (XML tags, weight 0.5), (2) direction validity (weight 0.5), (3) game reward (Δscore / max_tile, weight 2.0). Quality and length components were removed after the model reward-hacked the length penalty by generating filler text.

5. Experiments

5.1 Setup

All experiments ran on a single NVIDIA RTX 4060 Laptop GPU (8 GB VRAM), Ubuntu 22.04, CUDA 12.6. Classical agents were trained for 1M and 5M environment steps. LLM agents (Qwen2.5-0.5B and 1.5B) were trained for 300 GRPO prompts (∼12 hours). Agents were additionally evaluated in "hunt mode": unlimited stochastic episodes (ε=0.05) using the 1M-step checkpoint until reaching tile 2048 or exhausting 3,000 attempts.

5.2 Classical RL Results: 1M Steps

Table 1 presents all six classical agents evaluated over their last 100 training episodes at the 1M checkpoint.

DQN dominates at 1M steps, reaching tile 1024 in 10% of games. The 8× gap over PPO (6,384 vs. 1,038) is driven by experience replay: DQN's 100K buffer revisits rare high-scoring trajectories thousands of times, amplifying sparse reward signals (S&B §8.2). On-policy methods discard each trajectory after a single update.

Table 2: Classical Agent Performance (5M Steps)

Agent	Avg Score	Max Score	Max Tile	512+ Rate	1M→5M Δ	Verdict
DQN	**7,744**	**39,314**	**2048**	**76%**	**+174%**	Strong scaling
SARSA	2,456	11,436	1024	6%	+7%	Ceiling (§9.4)
A2C	1,944	9,640	1024	6%	+56%	Modest gain
SAC	1,255	7,882	512	<1%	+79%	Slow
QR-DQN	995	5,176	512	<1%	−9%	Degrading
PPO	962	4,900	512	<1%	−12%	Gets worse

Table 3: Hunt Mode Results (1M Checkpoint, Unlimited Attempts)

Agent	Attempts	Best Tile	Best Score	Verdict
DQN	3,902	**2048**	**21,572**	Only success
SARSA	3,000	1024	12,300	120-param ceiling
A2C	3,000	512	7,844	Stalls post-512
SAC	3,000	256	3,648	Entropy hurts late
PPO	193	64	520	Collapses early

Semi-gradient SARSA achieves second place (avg 2,455, max tile 1024) with only 120 parameters – 2,750× fewer than DQN's 330K. Its hand-crafted feature vector encodes monotonicity, corner strategies, merge potential, and snake patterns, providing the "strong inductive bias" that Sutton & Barto advocate (§9.5).

5.3 Classical RL Results: 5M Steps

Table 2 shows the same agents at the 5M-step checkpoint.

The 1M→5M scaling analysis reveals three distinct behaviors:

- **DQN scales strongly** (+174%), confirming that off-policy replay continues to benefit from more data accumulation.
- **SARSA reaches its representational ceiling** (+7%), consistent with the bounded approximation error under linear FA (S&B §9.4). The 120-parameter linear model cannot capture the complex tile interactions needed for tiles beyond 1024.
- **PPO and QR-DQN actively degrade** (−12% and −9% respectively). PPO develops degenerate tile-sliding loops; QR-DQN suffers from distributional overfitting with its 200-dimensional output head.

5.4 Hunt Mode (1M Checkpoint, Unlimited Attempts)

Each agent plays with stochastic action selection (ε=0.05) using its 1M-step checkpoint until reaching tile 2048 or exhausting 3,000 attempts.

DQN is the only agent to reach tile 2048 (attempt 3,902 of unlimited play, 0.026% rate). Hunt mode reveals that deterministic greedy policies create infinite cycles at board states where Q-values are poorly estimated; adding 5% exploration breaks these cycles. Semi-gradient SARSA's linear capacity ceiling (§9.4) prevents it from representing the complex tile interactions needed for the 1024→2048 merge.

PPO collapses to tile 64 in just 193 attempts – worse than random play – demonstrating that its policy has catastrophically overfit to a narrow set of states seen during 1M-step on-policy training.

Table 4: GRPO-Trained LLM Performance (300 Prompts)

Model	Avg Score	Max Tile	Params	ms/move
Qwen2.5-1.5B	**2,348**	**256**	1.8B	1,500
Qwen2.5-0.5B	704	64	494M	1,500

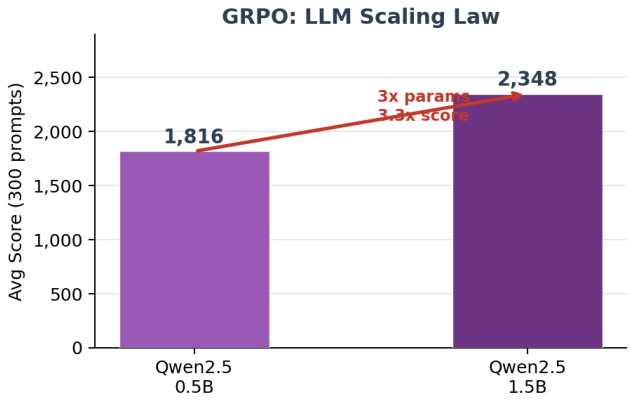

Figure 1: GRPO scaling: 3× parameters → 3.3× score.

5.5 RLVR Results: GRPO-Trained LLMs

Table 4 compares the two GRPO-trained LLM agents.

The 3× parameter increase (0.5B → 1.5B) yields a 3.3× score improvement (704 → 2,348), indicating that model scale is the primary lever for RLVR on spatial tasks. Figure 1 shows this scaling relationship.

Chain-of-thought reasoning. The 1.5B model produces interpretable move explanations. For example, given a board with 256 anchored in the bottom-right corner:

```
<think> 256 anchored bottom-right
-- good.  Gradient:  128, 64, 32,
16 intact along right/bottom edges.
RIGHT merges 4+4 in row 1 (+8).
Avoids scattering corner tile.
</think> <answer>RIGHT</answer>
```

Spatial reasoning failure. The 0.5B model reveals a fundamental limitation: it pattern-matches on token proximity rather than grid geometry. When tiles 4 appear at positions [0][1] and [1][0], the model incorrectly claims "4 at [0][1] and [1][0] are adjacent – LEFT merges them into 8." In reality, they are in different rows; LEFT moves each to column 0 independently with no merge. This spatial failure explains the 0.5B model's poor performance and suggests that larger models are needed for accurate mental simulation of board mechanics.

5.6 Classical RL vs. RLVR Comparison

Figure 2 directly compares all eight agents. Table 5 provides a dimension-by-dimension comparison.

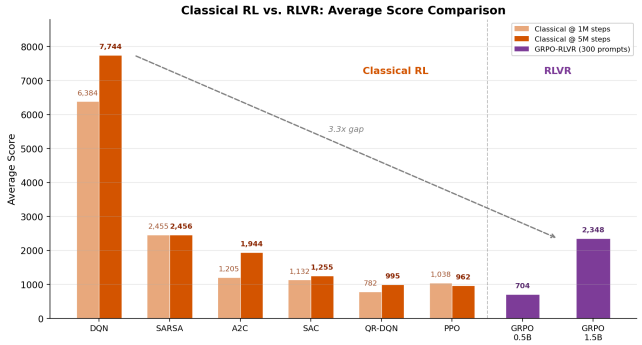

Figure 2: Classical RL vs. RLVR: average score comparison across 1M steps, 5M steps, and GRPO training (300 prompts).

Table 5: Head-to-Head: Classical RL vs. LLM-RLVR

Dimension	Classical RL (best)	LLM-RLVR (GRPO)
Best Avg Score	**7,744** (DQN, 5M)	2,348 (1.5B)
Best Max Tile	**2048** (DQN, hunt)	256 (1.5B)
Parameters	120 – 430K	494M – 1.8B
Inference	**<4 ms/move**	1,500 ms/move
Training data	5M env steps (∼2 hrs)	300 prompts (∼12 hrs)
Training compute	2 hrs (RTX 4060)	12 hrs (same GPU)
Interpretable	No (black box)	**Yes (chain-of-thought)**
Observation	**Tensor** (16ch)	ASCII text
Action space	Integer $\{0-3\}$	Natural language token

The 3.3× score gap between the best classical agent (DQN, 7,744) and the best RLVR agent (GRPO 1.5B, 2,348) reflects fundamental differences in data efficiency and representation. DQN processes 5M transitions with replay, revisiting each state thousands of times; GRPO sees only 300×4=1,200 board states. DQN's 16-channel tensor encoding is information-dense; the LLM must tokenize and parse ASCII text, introducing representational overhead.

However, RLVR provides a unique advantage: *interpretability*. Every GRPO move includes a chain-of-thought explanation that can be inspected, debugged, and corrected. Classical agents are complete black boxes. In applications where explainability is valued – educational tools, human-AI collaboration, safety-critical systems – the 400× latency penalty may be acceptable.

5.7 Training Dynamics

Figure 3 shows DQN's training curves over 5M steps, demonstrating steady improvement throughout. Figure 4 compares the scaling behavior from 1M to 5M steps across all classical agents.

6. Conclusion

We presented a systematic comparison of six classical RL agents and two GRPO-trained LLMs on the 2048 puzzle. Our controlled setup – shared CNN backbone, identical environment, three evaluation tiers (1M, 5M, hunt mode) – isolates algorithmic effects and yields four key findings:

Figure 3: DQN training curves over 5M steps. Score and max tile improve steadily, reaching 2048 tile and avg 7,744.

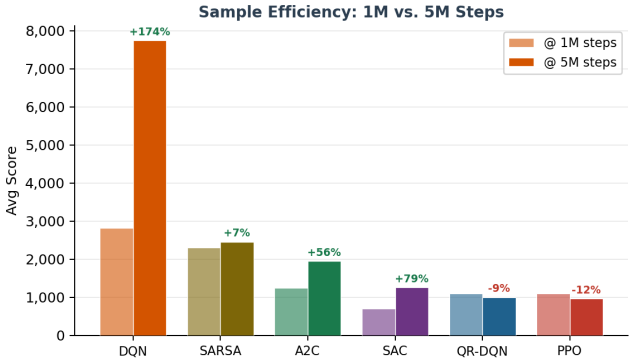

Figure 4: Sample efficiency: 1M vs. 5M steps. DQN benefits most (+174%); PPO and QR-DQN degrade (−12%, −9%).

1. **Off-policy replay is decisive.** DQN's 100K replay buffer (S&B §8.2) produces an 8× advantage over on-policy PPO on identical architectures (7,744 vs. 962). Experience replay amplifies sparse rewards that on-policy methods see exactly once and discard.
2. **Features beat deep networks at low data.** Semi-gradient SARSA with 120 hand-crafted parameters outperforms all deep on-policy agents (330K+ parameters), validating Sutton & Barto's thesis (§9.5) that domain-specific features provide stronger inductive bias than CNNs learning from scratch.
3. **More training actively hurts on-policy agents.** PPO and QR-DQN lose 9–12% score from 1M→5M steps, developing degenerate move loops. This is a cautionary finding for practitioners: longer training does not universally help.
4. **LLM scale drives RLVR performance.** 3× more parameters (0.5B → 1.5B) yields 3.3× higher score. GRPO uniquely produces interpretable per-move rationale, but spatial reasoning remains a bottleneck at small model scales.

Future work includes scaling GRPO to Qwen2.5-3B/7B on

an A100 GPU, replay buffer ablation (10K–500K) to quantify the exact contribution of replay to DQN's dominance, multi-seed evaluation for statistical significance, and hybrid LLM-MCTS approaches combining interpretability with search efficiency.

Acknowledgment

This project was completed as part of CS 5180 (Reinforcement Learning) taught by Professor Robert Platt at the Khoury College of Computer Sciences, Northeastern University. All experiments were conducted on a single NVIDIA RTX 4060 Laptop GPU (8 GB VRAM).

References

[1] D. Guo *et al.*, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *Nature*, vol. 645, pp. 633–638, 2025.

[2] Y. Wen *et al.*, "RLVR implicitly incentivizes correct reasoning," *arXiv:2506.14245*, 2025.

[3] P. Saligram, T. Bhathal, and R. Manihani, "2048: Reinforcement learning in a delayed reward environment," *arXiv:2507.05465*, 2025.

[4] L. Hu *et al.*, "lmgame-Bench: How good are LLMs at playing games?" *arXiv:2505.15146*, 2025.

[5] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.

[7] V. Mnih *et al.*, "Asynchronous methods for deep reinforcement learning," in *Proc. ICML*, pp. 1928–1937, 2016.

[8] W. Dabney, M. Rowland, M. Bellemare, and R. Munos, "Distributional reinforcement learning with quantile regression," in *Proc. AAAI*, 2018.

[9] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. ICML*, 2018.

[10] P. Christodoulou, "Soft actor-critic for discrete action settings," *arXiv:1910.07207*, 2019.

[11] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized language models," in *Proc. NeurIPS*, 2024.

[12] Z. Shao *et al.*, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv:2402.03300*, 2024.

[13] M. Szubert and W. Jaśkowski, "Temporal difference learning of n-tuple networks for the game 2048," in *Proc. IEEE CIG*, 2014.

[14] A. Raffin *et al.*, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.

[15] Qwen Team, "Qwen2.5: A party of foundation models," *arXiv:2412.15115*, 2024.

[16] D. Han and M. Han, "Unsloth," Available: `https://github.com/unslothai/unsloth`, 2024.

[17] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv:2407.17032*, 2024.

[18] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.